

**STRULE AUTOMATION**

Industrial Controls · Bay Area & Northern California

TRAINING PRIMER

PLC Basics for Maintenance Techs

I write these field notes for the people who actually keep a plant running: the maintenance leads, the techs, the operators who know every bearing and belt on the line but never had a reason to crack open a controls textbook. If you can read a print and swing a wrench, you already have most of what you need to understand a programmable logic controller (PLC). What follows is how I'd explain it standing next to you at the panel, coffee in hand, before we touch anything.

Why this matters on the floor

When a conveyor won't start and the operator swears the button is good, somebody has to figure out whether the problem is a switch, a wire, a jammed chute, or the logic itself. If the only person who can answer that is a contractor two hours away, your downtime clock is running the whole time you wait. A tech who understands what the PLC is doing can walk up, look at the right screen, and narrow the problem down in minutes. That's the whole point of learning this. You're not trying to become a programmer. You're trying to read the machine well enough to know where the trouble is.

What a PLC actually is

A programmable logic controller (PLC) is a ruggedized industrial computer built to sit in a hot, dusty, vibrating electrical panel and make decisions about your equipment, over and over, without crashing. It reads the state of things out in the plant, runs through a set of rules a controls engineer wrote, and then switches things on and off based on those rules.

Picture the old way: a wall of relays, timers, and hardwired push buttons, all clicking away to run a conveyor or a screen. A PLC replaces that wall of relays with a small box and a program. The physical wiring still exists for the field devices, but the "if this, then that" logic now lives in software you can view, change, and back up. That flexibility is the gift and the hazard. It's why a machine can be re-sequenced in an afternoon, and also why a single unseen change can bite you three shifts later.

The job it does on the line is simple to state: watch the inputs, decide, drive the outputs. Everything else is detail.

The scan cycle, and what "scan time" means

Here's the single most important idea in this whole primer. A PLC does not watch everything all at once, the way you might imagine. It works in a tight loop, over and over, and that loop is called the scan cycle.

Every pass through the loop has three basic steps:

1. **Read the inputs.** The PLC takes a snapshot of every input terminal: which push buttons are pressed, which limit switches are made, what the level sensor is reading. It freezes that picture in memory.
2. **Solve the logic.** It works through the program top to bottom using that frozen snapshot, deciding what every output should be.
3. **Write the outputs.** It updates all the output terminals at once, energizing motor starters, opening solenoids, turning on lights.

Then it goes right back to step one and does it again. And again. Hundreds of times a second on a typical machine.

The time it takes to make one full trip around that loop is the **scan time**, usually a few milliseconds on a typical machine (a millisecond, or ms, is one-thousandth of a second — check your controller's spec, because a big program on an older processor runs slower). Scan time matters because the PLC only "sees" the world at the start of each scan. If a signal blinks on and off faster than one scan, the PLC can miss it entirely. That's worth remembering when you're chasing a fast-acting proximity switch or a bounce on a contact that the operator sees but the logic doesn't.

Digital versus analog I/O

I/O just means input/output — the signals coming into the PLC and the signals going out of it. There are two flavors, and knowing which is which saves you a lot of confusion at the panel.

Digital I/O (also called discrete I/O) is on or off, one or zero, nothing in between. A start button is digital: pressed or not. A motor starter output is digital: energized or not. A full/not-full paddle switch, an e-stop, a zero-speed switch, a solenoid valve command — all digital. Most of what runs a conveyor line is digital.

Analog I/O is a smooth range of values, not just on and off. A level sensor in a surge bin reporting anywhere from empty to full, a pressure transmitter on a hydraulic pack, a variable-frequency drive (VFD) speed reference telling a motor to run at 40 percent — those are analog. Analog signals usually ride on a standard range like 4 to 20 milliamps (mA) or 0 to 10 volts (V); check your device's spec sheet for which. The PLC converts that current or voltage into a number it can do math with.

A quick plant example. On an inclined transfer conveyor, the pull-cord e-stop and the belt-drift switch are digital inputs, the motor run command is a digital output, and if that belt is driven by a VFD, the speed reference going out and the actual-speed feedback coming back are analog. One machine,

both flavors, and you'll troubleshoot each differently.

| Addresses and tags: the name on the wire

Every physical terminal on the PLC has an **address** — the controller's own coordinate for that point, something like `I:1/4` on an older platform or `Local:1:I.Data.4` on a newer one. Nobody wants to troubleshoot in that language, so on top of the address we put a **tag**: a plain-English name like `Conv_01_Start_PB` or `Surge_Bin_Level`.

The mapping between the physical terminal, its address, and its tag is the Rosetta Stone of the whole system. When the print says the start button lands on input terminal 4, and the program calls that point `Conv_01_Start_PB`, you now have a thread you can pull all the way from the button in the field to the rung in the logic. Good documentation keeps that thread intact. Half of real-world troubleshooting is just confirming that the wire, the address, and the tag all still agree with each other and with the print.

| Reading a ladder-logic rung

The most common PLC programming language is ladder logic, and it was drawn to look like the relay diagrams electricians already knew. Picture a ladder: two vertical rails with horizontal rungs between them. Power, conceptually, flows left to right across each rung. On the left side you have conditions (contacts). On the right side you have a result (a coil, the output). The question every rung asks is simple: **does this output turn on?**

Two symbols do most of the work:

- A **contact** represents an input condition, drawn as two vertical bars `] [`. A **normally-open (NO)** contact passes power when its signal is TRUE (the button is pressed, the switch is made). A **normally-closed (NC)** contact, drawn with a slash `] / [`, is the opposite: it passes power when its signal is FALSE, and blocks power when the signal goes TRUE.
- A **coil** represents the output, drawn as `( )` at the right end of the rung. If power makes it all the way across to the coil, the output turns on.

Two wiring ideas finish the picture:

- Contacts in **series** (one after another on the same line) are an **AND**. Every one of them must pass power for the output to come on.
- Contacts in **parallel** (stacked, offering two paths) are an **OR**. Any one path passing power is enough.

That's genuinely most of it. Series is AND, parallel is OR, NO passes when true, NC passes when false, and the coil comes on if power gets across.

A worked rung: starting a conveyor

Let me walk through a rung you'll see in some form on nearly every machine — a basic conveyor start with a start button, a stop button, and an interlock. In words, here's how it's wired:

- A **normally-open** contact for the start push button (`Conv_01_Start_PB`),
- in **parallel** with a normally-open contact for the running output itself (`Conv_01_Run`) — this is the seal-in, and I'll explain it in a second,
- and that pair in **series** with a contact for the stop push button (`Conv_01_Stop_PB`) — in the ladder this one is examined as **normally-open**, which is the opposite of what most people expect, so hold that thought,
- in series with a **normally-open** contact for the interlock, say an upstream permissive (`Upstream_Ready`),
- driving the coil `Conv_01_Run` .

Now read it the way the PLC does. Press the start button: the NO start contact passes power. The stop button is healthy and not pressed, so its input is energized and its normally-open contact is passing power. The upstream permissive is satisfied, so that NO contact is passing too. Power makes it across, the `Conv_01_Run` coil energizes, and the conveyor starts.

Here's the clever part. The moment `Conv_01_Run` turns on, its own NO contact — the one in parallel with the start button — closes. So when you let go of the start button, power still has a path through that sealed-in contact. The conveyor keeps running. That parallel contact is called a **seal-in** (or holding contact), and it's how a momentary push button gives you continuous run.

To stop, you press the stop button. That drops the stop input, its contact opens, power can no longer cross, the coil drops out, the seal-in opens, and everything resets. Same thing happens if the interlock drops: lose `Upstream_Ready` and the rung goes dead, exactly as it should.

Notice the safety logic baked in, and here's the part that trips people up. The stop push button out in the field is a **normally-closed** device: its contacts sit closed when nobody's touching it, so the PLC input stays energized the whole time the button is intact and unpressed. Pressing stop opens those contacts and drops the input. So does a broken wire or a backed-out terminal — the input dies exactly as if someone hit stop, and the conveyor won't run. A failure makes the machine safe, not dangerous. That's a deliberate design choice.

This is why, in the ladder, the stop is examined with a *normally-open* contact even though the field button itself is normally-closed. The energized input is what passes power, so the machine runs while the button is healthy, and anything that kills that input — a press, a broken wire, a loose terminal — opens the rung. Get that distinction straight and a lot of prints that looked contradictory suddenly make sense: the button says NC, the ladder shows a normally-open contact, and both are right. It's also why you never rewire a normally-closed field stop as normally-open to "make it simpler" — you'd be throwing away the fail-safe that protects whoever's downstream of that belt.

Online versus offline, and reading live rungs

There are two ways to look at a PLC program on your laptop.

Offline means you're viewing the saved copy of the program on your computer, disconnected from the controller. You can read it and study it, but you're looking at a file, not at what the machine is doing right now.

Online (monitoring) means your software is connected to the running controller and showing you live status. This is the mode that solves problems. When you're online, the contacts and coils that are currently passing power are highlighted — usually shown in a bright color or bold. You can literally watch the power flow across a rung in real time.

This is the fastest way to find out why something won't start. Bring up that conveyor rung online and look at which contacts are highlighted and which aren't. If the start button contact lights up when the operator presses it but the interlock contact stays dark, you've found it: the upstream permissive isn't satisfied. No guessing, no meter yet — the logic is telling you where power stops. That one skill, reading a live rung to see where the flow dies, is worth more on a bad night than anything else in this primer.

Forcing an I/O point

Now the sharp tool. **Forcing** means overriding an input or output in the PLC, telling it to ignore reality and treat a point as ON (or OFF) because you said so. Force a digital input ON and the logic behaves as if that switch is made, whether it is or not. Force an output ON and the PLC energizes it regardless of what the logic wants.

As a diagnostic, it's powerful. Suppose you suspect a proximity switch is bad. Force its input and watch whether the downstream logic finally lets the machine sequence — if it does, you've confirmed the switch or its wiring, not the program. Or force an output to a solenoid, with the equipment safely isolated, to prove the valve and coil respond. Used deliberately, forcing answers questions nothing else can.

And here is why it's dangerous, plainly. A force overrides reality. When you force an output ON, that motor or valve can move no matter what the safety logic is trying to do. When you force an input, you have blinded the PLC to what's actually happening in the field — you've told it a guard is closed or a bin isn't full when it may well be the opposite. A forced point leaves a live plant in a non-normal state, and worst of all, forces are invisible to the next person. The operator sees a machine that looks normal and has no idea a switch has been overridden. People have been hurt by a force somebody left in "just for a minute" and forgot.

So the rules I hold to, and ask everyone I train to hold to: force only with a clear purpose, only with the affected equipment isolated or with everyone clear and warned, one point at a time, and never, ever walk away leaving a force in place. Remove it the moment the test is done and confirm it's gone. Most

controls software will flash a warning that forces are active; treat that warning as a live hazard, because it is. Forcing is a thing you do for sixty seconds under full attention, not a thing you leave running.

Back up the program before you touch anything

One more idea that saves careers. **The running controller and the saved program on the shelf are not the same thing.** The PLC is executing whatever is in its memory right now, which may or may not match the file somebody archived two years ago. If nobody has captured the live program lately, and a processor dies or a change goes wrong, you can lose the exact logic that runs your plant — and rebuilding it from memory and prints is a nightmare I've watched plants live through.

So before you change one rung, one setpoint, one anything: go online and **upload** the program from the running controller to your laptop, and save it with a clear name and date. That's your known-good starting point. If your edit makes things worse, you have something to fall back to. Capture first, change second. It costs you five minutes and it's the cheapest insurance in this trade.

Try this on your own plant (safely)

Pick one conveyor you know well. With the machine running normally and safe to observe, get someone who has the controls software and access to bring up that conveyor's start rung online. Just watch it. Have the operator start and stop the conveyor and see the contacts light up and drop out in real time. Trace the seal-in. Find the interlock contact and figure out what real-world condition satisfies it. You don't change a thing — you're building the mental map of how this one rung mirrors the machine in front of you. Do that on three machines and you'll never look at a stalled conveyor the same way again.

A word on the hands-on side. Watching a live rung is one thing; putting hands on a jammed chute or a suspect switch is another. Before any hands-on work, apply lockout/tagout (LOTO) and prove zero energy — verify the machine is truly de-energized and can't start, don't just trust the panel light. On any VFD or drive, respect the stored DC-bus energy: those capacitors hold a dangerous charge for minutes after power-down, so wait out the drive's rated discharge time and verify before you reach in. Any live electrical reading means arc-flash exposure, so wear the right personal protective equipment (PPE) for that panel and follow your site's energized-work rules. And when you restore a machine, prove the interlocks and e-stops actually work before you hand it back — press the stop, drop the guard, confirm the machine responds. An interlock you assumed was good and never tested is just a story until the day it fails.

If your plant runs biogas — organics or anaerobic digestion (AD) — the same care extends to the atmosphere. Hydrogen sulfide (H₂S) reaches its immediately-dangerous-to-life-or-health (IDLH) level around 100 parts per million (ppm) and deadens your sense of smell before it kills you, so never trust your nose. Methane is explosive between roughly 5 and 15 percent in air (its lower and upper explosive limits, LEL and UEL). Digesters and many pits are permit-required confined spaces — never

enter on your own judgment, and carry a calibrated 4-gas monitor. Wiring in those areas falls under hazardous-area classification (Class I, Division ratings per the National Electrical Code, NEC 500/505); it's built a specific way for a reason, so don't modify it casually.

Where a second set of eyes helps most

Everything above is the foundation these plants run on, and most techs pick it up faster than they expect once someone shows them a live rung. The place I see trouble is at the seams — where one integrator installed the PLC, another rewired half the I/O during an expansion, and the backup on the shelf no longer matches the controller. If your logic and your prints and your running program have drifted apart across vendor hand-offs, that's exactly where an independent read-through pays for itself, and it pairs naturally with the migration and commissioning checklists that go with this primer.

Glossary

- **Programmable logic controller (PLC):** Industrial computer that reads inputs, runs logic, and drives outputs to control equipment.
- **Scan cycle:** The repeating loop the PLC runs — read inputs, solve logic, write outputs — over and over.
- **Scan time:** How long one full scan cycle takes, typically a few milliseconds; the PLC only sees the world once per scan.
- **Millisecond (ms):** One-thousandth of a second.
- **I/O (input/output):** The signals coming into and going out of the PLC.
- **Digital (discrete) I/O:** A signal that is only on or off — buttons, limit switches, motor run commands.
- **Analog I/O:** A signal that varies over a range — level, pressure, speed reference; often 4–20 mA or 0–10 V.
- **Address:** The controller's own coordinate for a physical terminal (e.g., `I:1/4`).
- **Tag:** A plain-language name mapped to an address (e.g., `Conv_01_Start_PB`).
- **Ladder logic:** The relay-style programming language; rungs of contacts driving coils.
- **Contact:** A symbol for an input condition; passes or blocks power.
- **Normally-open (NO) contact:** Passes power when its signal is true.
- **Normally-closed (NC) contact:** Passes power when its signal is false.
- **Coil:** The output symbol at the right of a rung; turns on if power reaches it.
- **Series (AND):** Contacts in a line; all must pass power.
- **Parallel (OR):** Contacts stacked; any path passing power is enough.
- **Seal-in (holding contact):** A parallel contact of the output itself that keeps a rung energized after a momentary start button is released.

- **Online / monitoring:** Connected to the running controller, viewing live rung states.
- **Offline:** Viewing the saved program file, disconnected from the controller.
- **Forcing:** Overriding an I/O point to a fixed state, ignoring reality; a powerful diagnostic and a serious hazard if left in place.
- **Upload:** Copying the live program out of the running controller to your computer to save it.
- **Variable-frequency drive (VFD):** A drive that varies motor speed; holds dangerous DC-bus charge after power-down.
- **Lockout/tagout (LOTO):** Isolating and securing energy so equipment cannot start during hands-on work.
- **PPE:** Personal protective equipment, including arc-flash gear for energized work.
- **H₂S, LEL/UEL, IDLH, ppm:** Hydrogen sulfide; lower/upper explosive limits; immediately-dangerous-to-life-or-health level; parts per million — biogas hazard terms.
- **AD:** Anaerobic digestion.
- **NEC:** National Electrical Code, including hazardous-area classification (Class I, Div per NEC 500/505).